

AI Tokenomics for Software Engineering: A Practical Economic Model for Iterative Model Routing

Dominik Gorecki

March 2026

Abstract

This paper develops a practical tokenomics framework for model routing in software engineering (SWE) workflows. We treat tokens as direct spend and explicitly account for downstream costs from incorrect outputs. Work is decomposed as $W = (W_c, W_s)$, where W_c represents context complexity (input and reasoning burden) and W_s represents output scope (code delta). For each step, the router selects a model from the Pareto frontier by minimizing expected economic cost, combining inference cost with the expected cost of errors through an engineer-calibrated price-of-error term $\lambda_E(W)$. We then extend this objective to iterative SWE loops using a recursive value function that re-routes at each residual task state. The formulation is implementation-ready and supports both human-in-the-loop and LLM-driven repair paths.

1 Introduction

Production SWE agents do not incur only token costs. A weak first pass can trigger additional model calls, test failures, delayed merges, human review overhead, or incidents. A useful routing objective must therefore optimize total expected economic cost, not token spend alone.

Recent token accounting work in agentic software engineering shows that cost is distributed across planning, generation, retries, and repair loops rather than concentrated in a single call [3]. Prior economic LLM evaluation work formalized explicit error-cost-aware objectives [1, 2], while routing research established cost-aware cascades that can jointly optimize quality and spend [4]. This paper specializes and extends those ideas for iterative SWE execution by introducing:

1. explicit task decomposition $W = (W_c, W_s)$,
2. a calibrated $\lambda_E(W)$ that includes both human and LLM-based repair,
3. a per-step economic objective, and
4. a recursive value function for multi-iteration routing.

2 Related Work

Three lines of prior work motivate this formulation. First, economic evaluation frameworks for LLMs formalize metrics that combine inference cost with failure consequences, directly motivating our use of a calibrated price-of-error term [1, 2]. Second, cost-aware model routing and cascading strategies show that frontier-based selection can improve cost-performance trade-offs relative to single-model policies [4]. Third, recent SWE-focused tokenomics evidence characterizes where token spend concentrates in multi-step agentic workflows, motivating explicit treatment of iterative repair and residual work [3]. Our contribution is to integrate these strands into a state-based, recursive router specialized for software engineering tasks.

3 LLM Pareto Frontier for SWE Routing

Let M be the available model set. For a given task state W , each model $m \in M$ has:

- direct inference cost $M_c(m, W)$, and
- first-pass success probability $A(m, W)$.

Model m is Pareto-dominated if there exists $m' \in M$ such that

$$M_c(m', W) \leq M_c(m, W), \quad A(m', W) \geq A(m, W),$$

with at least one strict inequality. The non-dominated set $\mathcal{P}(W) \subseteq M$ is the routing frontier.

In practice, routing over $\mathcal{P}(W)$ removes clearly suboptimal options and stabilizes policy behavior when model catalogs are large and frequently updated, consistent with prior cascade-style routing practice [4].

Figure 1 gives an illustrative accelerating-decay frontier. The curve is not a measurement claim; it is a visual aid showing a stylized trade-off where better capability tends to come at lower marginal cost reductions. We place leading models on the curve, dominated models inside the curve, and one breakthrough model outside the curve.

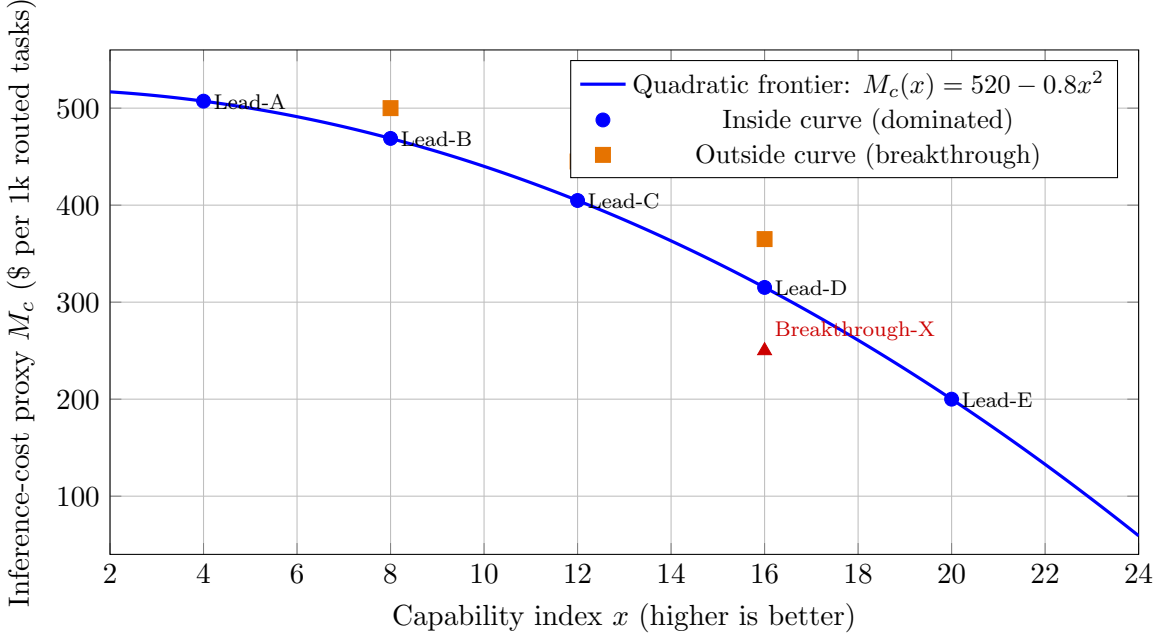


Figure 1: Illustrative Pareto geometry: leading models lie on the frontier, dominated options lie inside it, and a breakthrough model can appear outside (below) the old frontier.

In this framework, routing focuses on models on the frontier (the efficient set). Models inside the curve are dominated and should be selected rarely, if ever, once M_c and A are estimated correctly. A stable model outside the curve (as in Breakthrough-X) means the previous frontier is outdated: either the old fit “breaks” for current market data or a genuine technical advancement has shifted the frontier. In that case, the new outside model may be chosen much more often than interior models and often displace older frontier members.

4 Task Decomposition: W_c and W_s

We represent SWE work as a state variable with two components:

$$W = (W_c, W_s).$$

Context complexity W_c . This captures input and reasoning burden, e.g., repository breadth touched, dependency depth, ambiguity, prior failing traces, and expected total reasoning/context tokens.

Output scope W_s . This captures requested change magnitude. A practical default is code delta:

$$W_s \equiv \Delta \text{LoC} = \text{LoC}_{\text{added}} + \text{LoC}_{\text{removed}}.$$

This decomposition is operational: both $M_c(m, W)$ and $A(m, W)$ should be estimated as functions of (W_c, W_s) rather than from a single scalar task-size proxy.

Illustrative task examples.

1. **High W_c , low W_s .** A task requires broad repository traversal, reading many pattern/configuration files, understanding deep dependencies, and external research to resolve ambiguity—but the final code change is only a few lines.
2. **Low W_c , high W_s .** A task is conceptually simple and needs little context (e.g., a clear mechanical migration), but requires many edits across files, with large added/removed LoC.

Both can correspond to high overall work W under a composition function $W = f(W_c, W_s)$, even though the source of difficulty differs. In this model, W_c and W_s are treated as distinct dimensions, but in practice they are often positively correlated.

5 Defining and Interpreting the Cost of Error (λ_E)

5.1 Definition

The Cost of Error, $\lambda_E(W)$, is measured in *dollars per failed model attempt* for task state W . It converts an imperfect output into expected downstream business cost and follows the economic-evaluation framing in prior work [1, 2]. In iterative routing, the same form is re-evaluated at each step using the current residual state (e.g., $\lambda_E(W_t)$ at iteration t).

A practical decomposition is:

$$\lambda_E(W) = r_{\text{eng}} \mathbb{E}[t_{\text{human}} \mid W] + \mathbb{E}[C_{\text{LLM-repair}} \mid W] + c_{\text{inc}} \Pr(\text{incident} \mid W) + c_{\text{delay}} \mathbb{E}[d_{\text{delay}} \mid W].$$

where:

- $\lambda_E(W)$ is expected cost of one failed model attempt at task state W ,
- r_{eng} is loaded engineering cost (\$/hour),
- $\mathbb{E}[t_{\text{human}} \mid W]$ is expected human remediation time after a failed attempt,
- $\mathbb{E}[C_{\text{LLM-repair}} \mid W]$ is expected follow-up LLM repair spend,
- c_{inc} is average monetary impact per incident,
- $\Pr(\text{incident} \mid W)$ is the probability a failed attempt triggers an incident,
- c_{delay} is monetary cost per unit delay (e.g., per day),
- $\mathbb{E}[d_{\text{delay}} \mid W]$ is expected delay caused by a failed attempt.

5.2 Why this includes LLM-based fixing

If regressions are fixed by another model pass, that does *not* make error cost zero. It shifts cost from purely human labor into a mixed repair path.

Accordingly, $\lambda_E(W)$ includes:

- expected follow-up model tokens for repair attempts,
- validation/retest cost,
- residual human escalation when automated repair fails,
- expected incident and delay losses.

So $\lambda_E(W)$ is not just a reporting metric. It is a decision variable that determines when paying more upfront for a stronger model reduces total expected spend.

5.3 Loaded engineering cost example

The loaded engineering rate r_{eng} is all-in cost per productive engineering hour (salary, benefits, overhead, tooling).

$$r_{\text{eng}} = \frac{\text{annual all-in engineering cost}}{\text{productive engineering hours/year}}.$$

Example: if annual all-in cost is \$360,000 and productive hours are 1,800, then $r_{\text{eng}} = \$200/\text{hour}$. If a failed output induces 2.5 hours of human correction, that component contributes \$500 to $\lambda_E(W)$ before incident and delay terms.

6 Per-Step Objective

For each routing decision, choose model m minimizing expected immediate economic cost:

$$C(m, W) = M_c(m, W) + \lambda_E(W) (1 - A(m, W)).$$

Interpretation:

- $M_c(m, W)$ is direct token-priced inference cost,
- $1 - A(m, W)$ is failure probability,
- their product with $\lambda_E(W)$ is expected downstream correction loss.

7 Recursive Value Function for Iterative SWE

The recursive value function $V(W)$ represents the minimum expected total economic cost to complete the remaining work from state W . Unlike a one-step objective, it accounts for both immediate routing cost and the expected future cost after failures, including additional repair or retry steps. This makes the policy dynamic: each iteration updates the residual state and re-optimizes model choice based on the new expected cost-to-go.

7.1 Simplified form

A compact expression is:

$$V(W) = \min_{m \in \mathcal{P}(W)} C(m, W).$$

This is suitable when each step is treated independently.

7.2 Full iterative form

For agentic SWE loops, failed attempts leave residual work W' . The full Bellman-style objective is:

$$V(W) = \min_{m \in \mathcal{P}(W)} \left[M_c(m, W) + (1 - A(m, W)) (\lambda_E(W) + \mathbb{E}[V(W') \mid m, W]) \right],$$

with boundary condition $V(0) = 0$.

Component interpretation:

- $V(W)$: minimum expected total remaining cost-to-completion from state W .
- $\min_{m \in \mathcal{P}(W)}$: choose the frontier model that minimizes expected total cost.
- $M_c(m, W)$: immediate inference spend for model m at the current state.
- $1 - A(m, W)$: probability the current attempt fails with model m .
- $\lambda_E(W)$: expected downstream correction cost triggered by a failed attempt at state W .
- $\mathbb{E}[V(W') \mid m, W]$: expected future cost-to-go after failure and transition to residual state W' .

Operationally, the equation balances short-term and long-term economics. A stronger model may have higher immediate cost $M_c(m, W)$ but lower failure probability, which can reduce both immediate error cost $\lambda_E(W)$ and expected future cost $\mathbb{E}[V(W') \mid m, W]$. Conversely, a cheaper model is optimal only when its extra failure risk does not increase downstream iterative cost enough to outweigh upfront savings.

This explicitly captures per-iteration re-routing: after each failure, update the task state estimate and solve the minimization again.

8 How to Operationalize the Router

This section translates the economic framework into an implementation workflow. The goal is to move from theory to a deployable routing policy by estimating each model component from production traces, learning residual-task dynamics, and enforcing practical guardrails so routing remains cost-efficient and reliable in real SWE loops.

1. Estimate $M_c(m, W)$ from token logs and pricing tables, using workload-level token traces as in SWE tokenomics analyses [3].

2. Estimate $A(m, W)$ from task outcomes by model and task class.
3. Fit $\lambda_E(W)$ components using observed repair paths (LLM-only, mixed, human-escalated).
4. Learn transition distribution $p(W' | m, W)$ from residual-diff traces.
5. Route over the Pareto frontier with safety constraints (minimum quality floor, retry cap, escalation trigger).

9 Evaluation Plan

This evaluation should be run as a controlled, paired comparison across routing policies. For each feature implementation, user request, or new application class, construct the same initial task state W and run all policies against that exact W (same repo snapshot, prompt context, and acceptance criteria). Because outcomes are stochastic, repeat each policy multiple times per task (e.g., with different seeds/sampling draws) and compare distributions, not single runs.

A first empirical study should include the following tests:

1. **Always-cheap model baseline.** Route every attempt to the lowest-cost model. This measures the downside of minimizing immediate token spend while ignoring downstream error costs.
2. **Always-strong model baseline.** Route every attempt to the strongest model. This measures the upside of maximizing per-attempt quality at higher direct inference cost.
3. **Static two-stage cascade baseline.** Apply a fixed policy (e.g., cheap first, strong on failure) without state-dependent re-optimization. This measures gains from simple cascades versus adaptive routing.
4. **Recursive economic router (proposed).** Re-optimize at each residual state using the value function objective. This tests whether dynamic, state-aware routing reduces total cost-to-completion.

Evaluation protocol:

- run each policy multiple times on the same W to estimate variance,
- aggregate results per task and across tasks,
- report both central tendency (mean/median) and dispersion (e.g., p90/p95 or confidence intervals).

Primary metrics: total expected dollar cost per completed task, success rate at a fixed quality bar, number of retries to completion, and p95 latency.

10 Conclusion

Token spend alone is insufficient for SWE model routing. This paper frames routing as an expected total-cost problem by combining direct inference spend with a per-attempt Cost of Error $\lambda_E(W)$, then extending that objective through a recursive value function for iterative repair loops. The practical implication is clear: model selection should be re-optimized at each residual task state, not fixed once at the start. With paired evaluations on identical task states W across baseline and adaptive policies, teams can quantify when stronger upfront routing reduces downstream retries and incident risk, yielding lower total economic cost-to-completion.

References

- [1] Werner Zellinger and Craig Thomson. *Economic Evaluation of LLMs*. arXiv:2507.03834, 2025. Abstract; PDF; HTML.
- [2] Erol et al. *Cost-of-Pass: An Economic Framework for Evaluating Language Models*. arXiv:2504.13359, 2025. Abstract; PDF.
- [3] Ali Salim, Marc Latendresse, Hoda Khatoonabadi, and Emad Shihab. *Tokenomics: Quantifying Where Tokens Are Used in Agentic Software Engineering*. arXiv:2601.14470, 2026. Abstract; PDF.
- [4] Lingjiao Chen, Matei Zaharia, and James Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. arXiv:2305.05176, 2023. Abstract; PDF.